

<College / University Name>

A Final-Year B.Tech Major Project Report

PROJECT TITLE

UniVerify V2.0

A Decentralized Academic Certificate Verification Protocol Using Alphanumeric Keyspace Mapping

SUBMITTED BY

<Student Name>

Reg. No.: <2021CSE101>

B.Tech — Computer Science & Engineering

UNDER THE GUIDANCE OF

<Guide Name>

Designation: <Assistant Professor>

Department of CSE

Solidity ^0.8.0 • Ethers.js v5.7.2 • MetaMask • Ethereum Sepolia Testnet

Academic Year <2024–25>

Abstract

Traditional academic credential verification depends on centralized relational databases and manual, paper-based pipelines that suffer from high latency (7–14 days), recurring administrative cost, a single point of failure (SPOF), and easy forgery of unanchored documents.

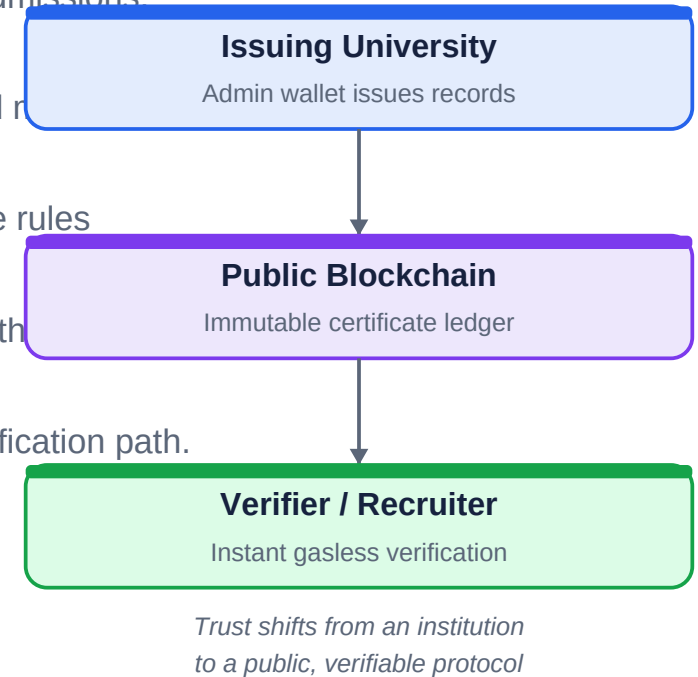
UniVerify V2.0 is a decentralized web application that anchors academic certificate records onto the public Ethereum Sepolia Testnet, making them immutable, globally accessible and verifiable in near real time. The smart contract binds each record to a student's unique alphanumeric Registration Number using a string-keyed state mapping, delivering $O(1)$ constant-time lookups and Role-Based Access Control (RBAC) that restricts write operations to the deployer (University Admin) wallet. Verification is a gasless, read-only view call executed client-side via Ethers.js and MetaMask.

SCOPE (PHASED)

Phase 1 (current, third year): a Production Ledger Prototype operated under a Manual Administrative Curation paradigm — the admin enters records in the Remix IDE and the hash field holds a manually-typed alphanumeric mock placeholder. Phase 2 (final year): an autonomous AI agent computes deterministic SHA-256 / IPFS hashes and broadcasts transactions programmatically.

Introduction

- **Problem context.** Academic credential fraud and slow manual verification cause real losses in admissions, recruitment and immigration screening.
- **Blockchain / DLT.** A decentralized, append-only ledger removes the central trusted authority and makes records tamper-evident.
- **Smart contracts.** Self-executing code on the Ethereum Virtual Machine (EVM) enforces issuance rules deterministically at the protocol level.
- **Decentralized identity.** Credentials are anchored on-chain and verified by anyone, anywhere, without the issuing institution.
- **Our contribution.** A string-keyed certificate registry with $O(1)$ lookups, RBAC, and a gasless verification path.



Literature Survey

Representative foundational works — replace/extend with your cited papers.

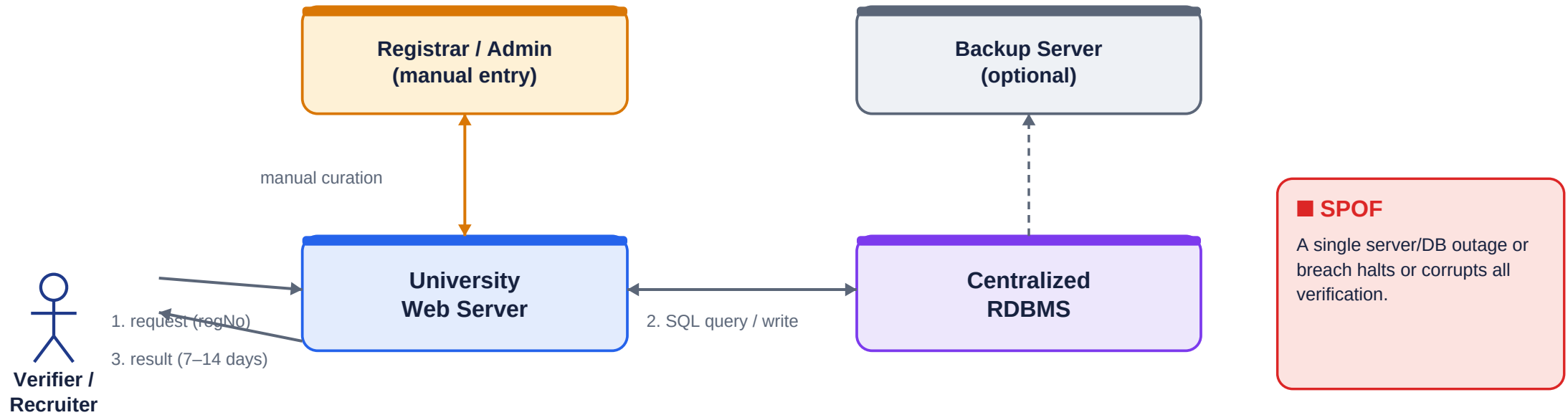
Ref.	Author (Year)	Focus / Contribution	Limitation
[1]	Nakamoto (2008)	Bitcoin: a peer-to-peer electronic cash system; first decentralized, tamper-evident ledger.	Not programmable; no native support for arbitrary credential records.
[2]	Buterin (2014)	Ethereum: a Turing-complete smart-contract platform enabling programmable trust on the EVM.	State-changing writes cost gas; throughput / scalability limits.
[3]	Benet (2014)	IPFS: content-addressed, distributed file storage referenced by cryptographic hash.	Availability depends on pinning; not a verification protocol by itself.
[4]	MIT Media Lab — Blockcerts (2016)	Open standard for issuing and verifying blockchain-anchored academic certificates.	Relies on issuer infrastructure; anchoring/verification overhead.
[5]	Grech & Camilleri (2017)	EU JRC study on blockchain in education; surveys credentialing use-cases.	Highlights governance, scalability and adoption challenges.

Existing System

- **Centralized RDBMS.** Each institution stores degree records in a private relational database hosted on its own servers.
- **Manual / paper verification.** Verifiers email or call the registrar; officials manually search records and issue confirmation letters.
- **Third-party agencies.** Background-verification companies act as paid intermediaries, adding cost and further delay.
- **Standalone digital PDFs.** Some institutions issue signed PDF certificates that are easy to copy and visually manipulate.
- **Trust model.** Verification trust rests entirely on the availability and integrity of one central authority and its servers.

Architecture of Existing System

Centralized request-response model with a single point of failure.



Drawbacks & Problem Statement

High latency

Manual lookups take 7–14 days, delaying admissions and hiring.

Single point of failure

Central server/DB outage or breach stops everything.

Administrative cost

Repeated manual audits create recurring overhead.

Credential forgery

Unanchored PDFs are trivially copied or altered.

PROBLEM STATEMENT

Design a verification protocol that eliminates the centralized single point of failure and manual latency of conventional systems by anchoring tamper-evident certificate records on a public blockchain — enabling instant, constant-time, globally accessible verification while restricting issuance to an authorized institutional authority.

Proposed System

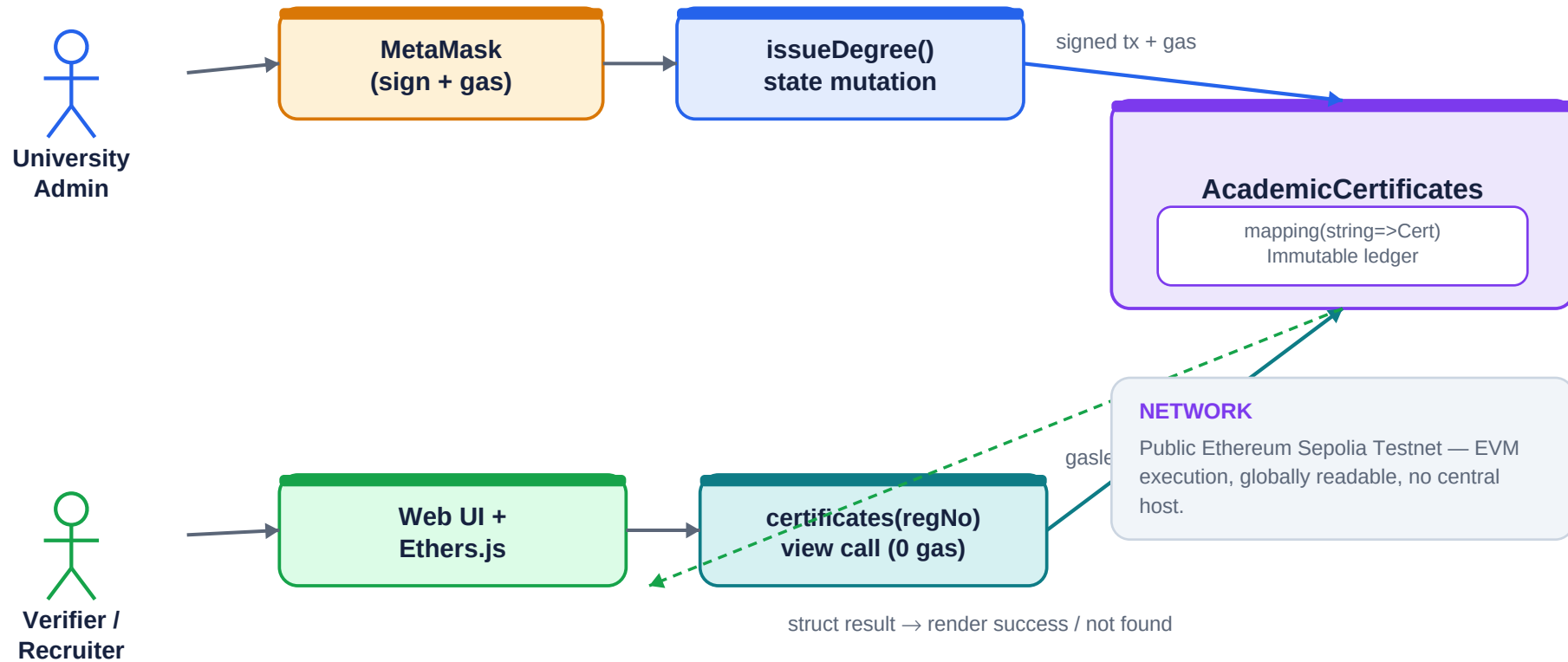
- **On-chain anchoring.** Certificate records are written to a Solidity smart contract on the Ethereum Sepolia Testnet — immutable once mined.
- **Alphanumeric keyspace mapping.** `mapping(string => Certificate)` keyed by Registration Number for efficient retrieval.
- **Role-Based Access Control.** The constructor freezes the deployer as the sole admin; unauthorized users cannot interact with the EVM layer.
- **Gasless verification.** Verifiers call a read-only view function via Ethers.js — instant and free of gas.
- **Safety filter.** Zero-state (blank) results are intercepted and shown as “Record Not Found”.

Immutable**O(1) lookup****No SPOF****Gasless reads****RBAC-secured****Global access****PHASE NOTE**

Phase 1 uses manual entry in Remix with a mock hash; real SHA-256 / IPFS hashing arrives in Phase 2.

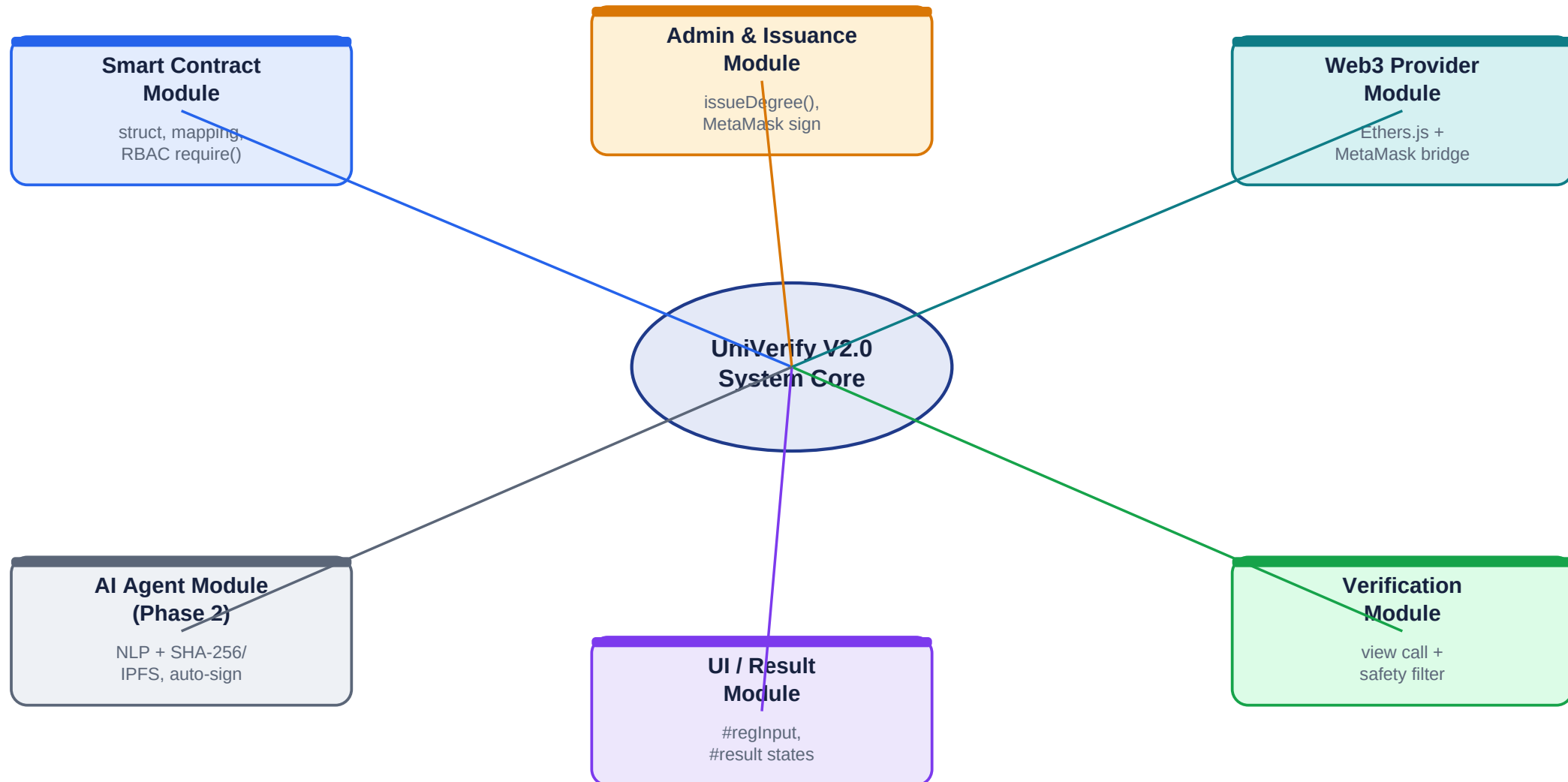
Architecture of Proposed System

Write path (admin, costs gas) and read path (verifier, gasless).



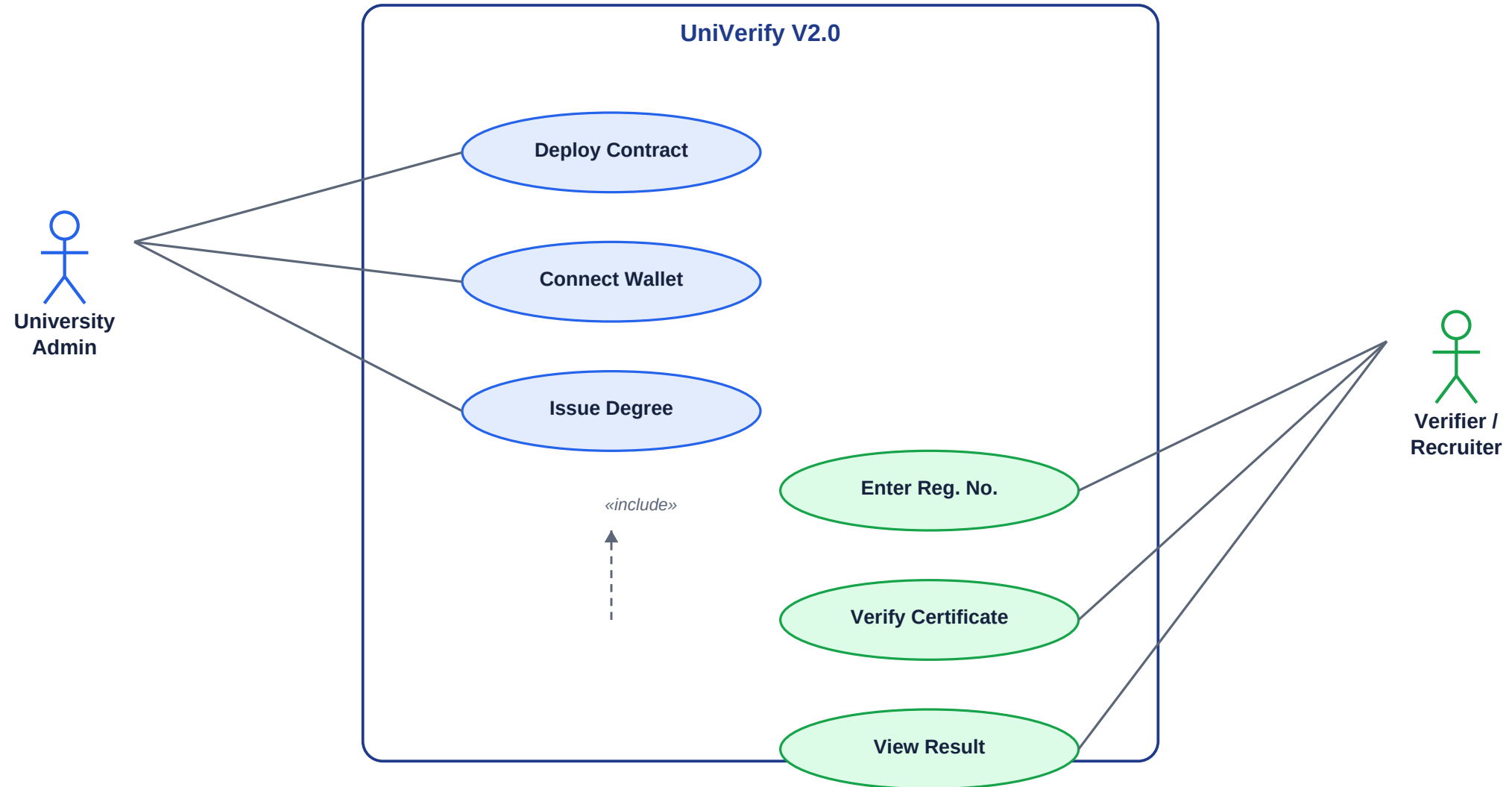
Module Diagram

Hub-and-spoke decomposition of the UniVerify V2.0 system.



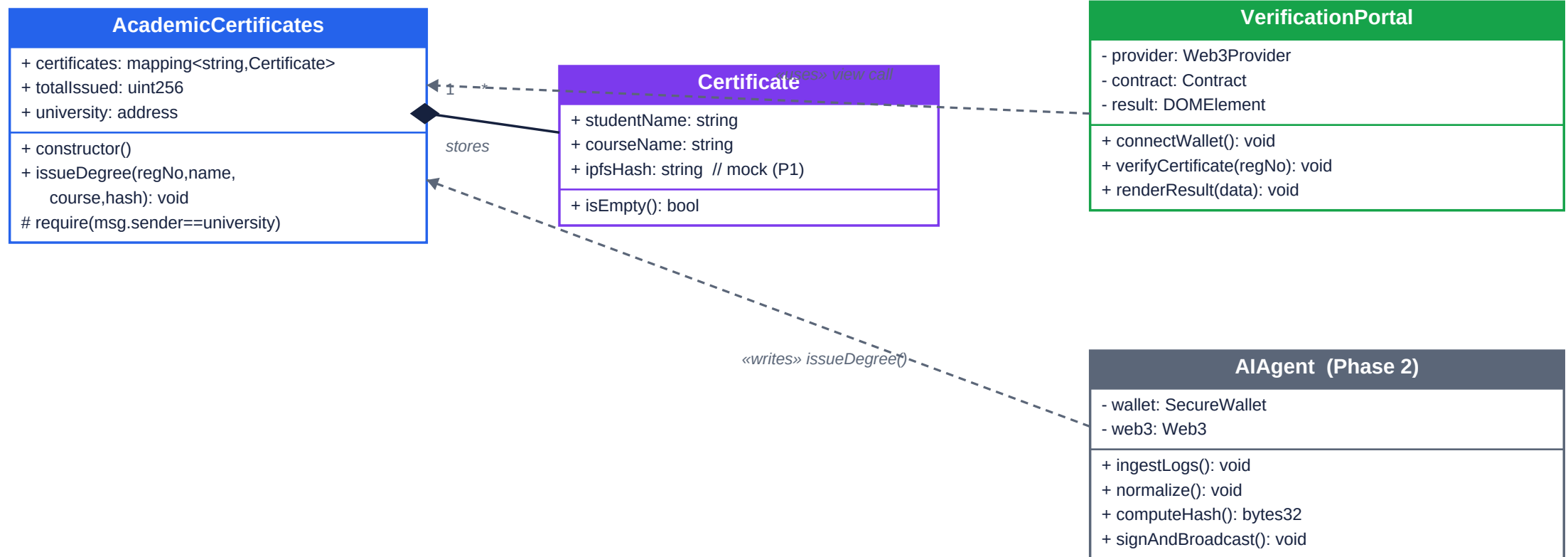
Use Case Diagram

Actors and their interactions with the verification system.



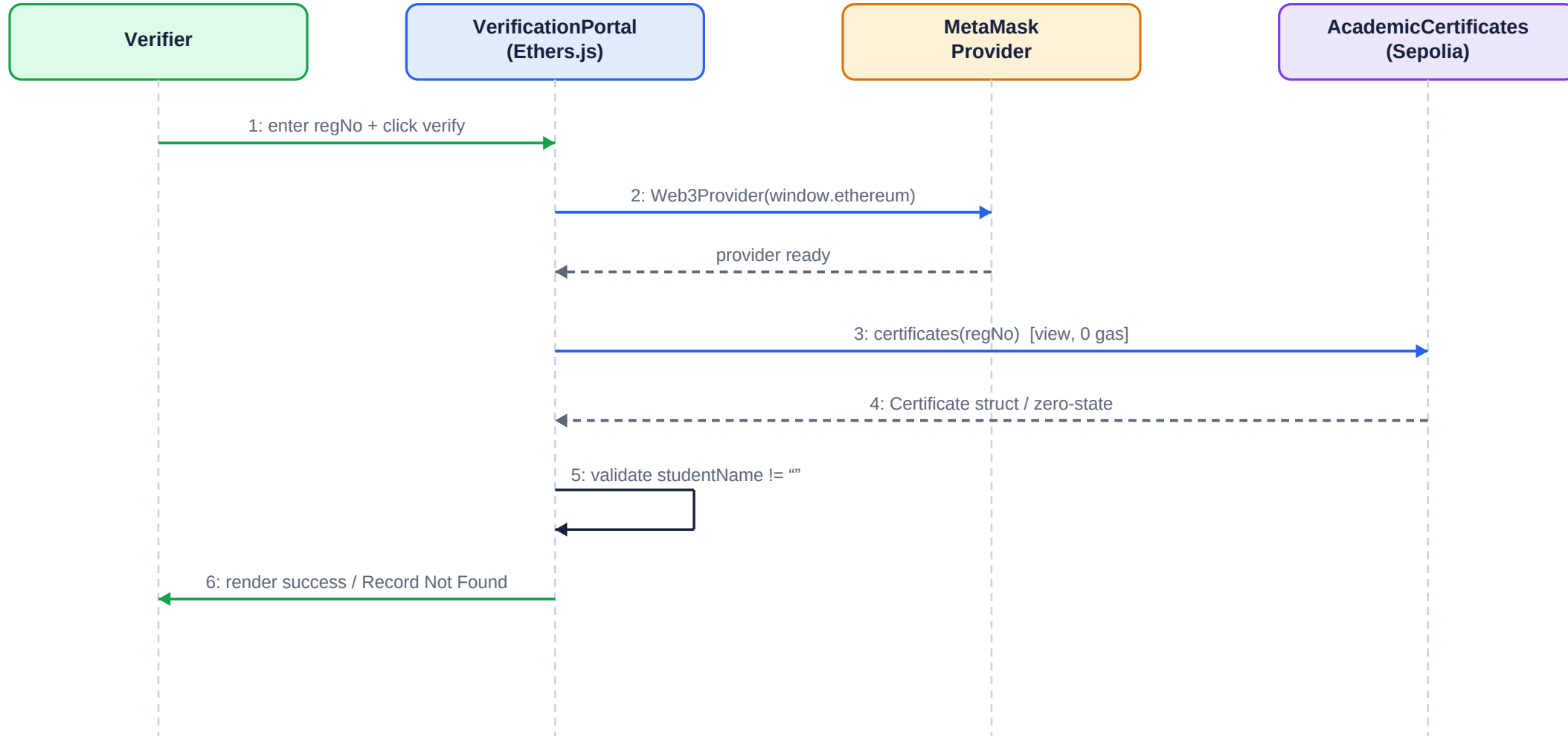
Class Diagram (Model Classes)

Core domain model: contract, record struct, frontend and Phase-2 agent.



Sequence Diagram (Verification)

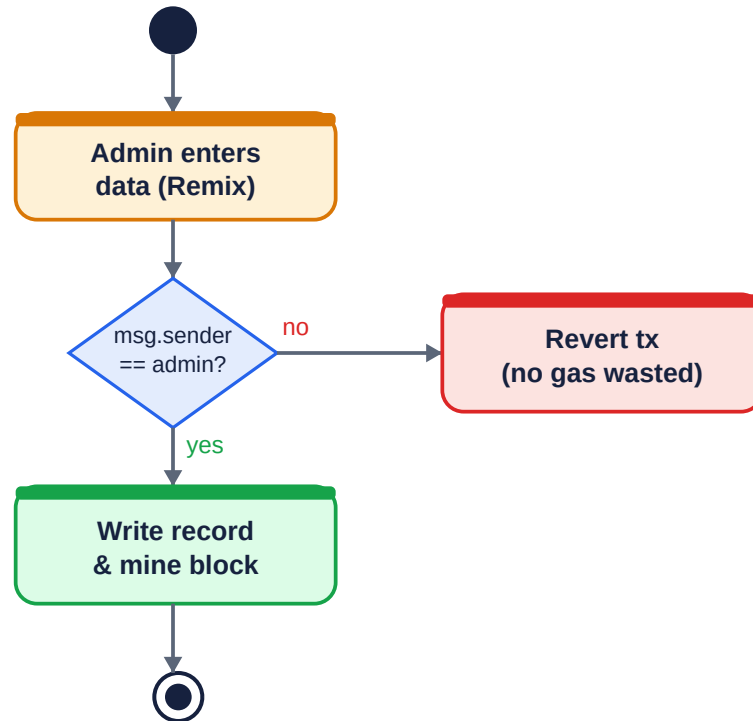
Gasless read-path message flow between client and contract.



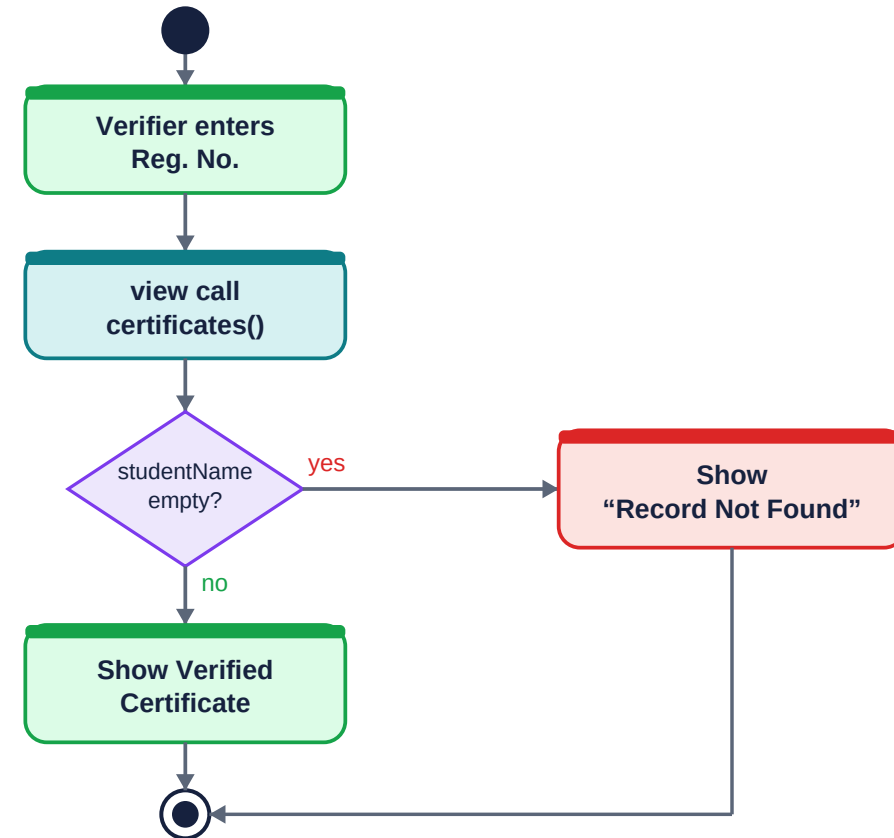
Activity Diagram

Issuance (write) and verification (read) control flow with a decision.

Issuance (write path)

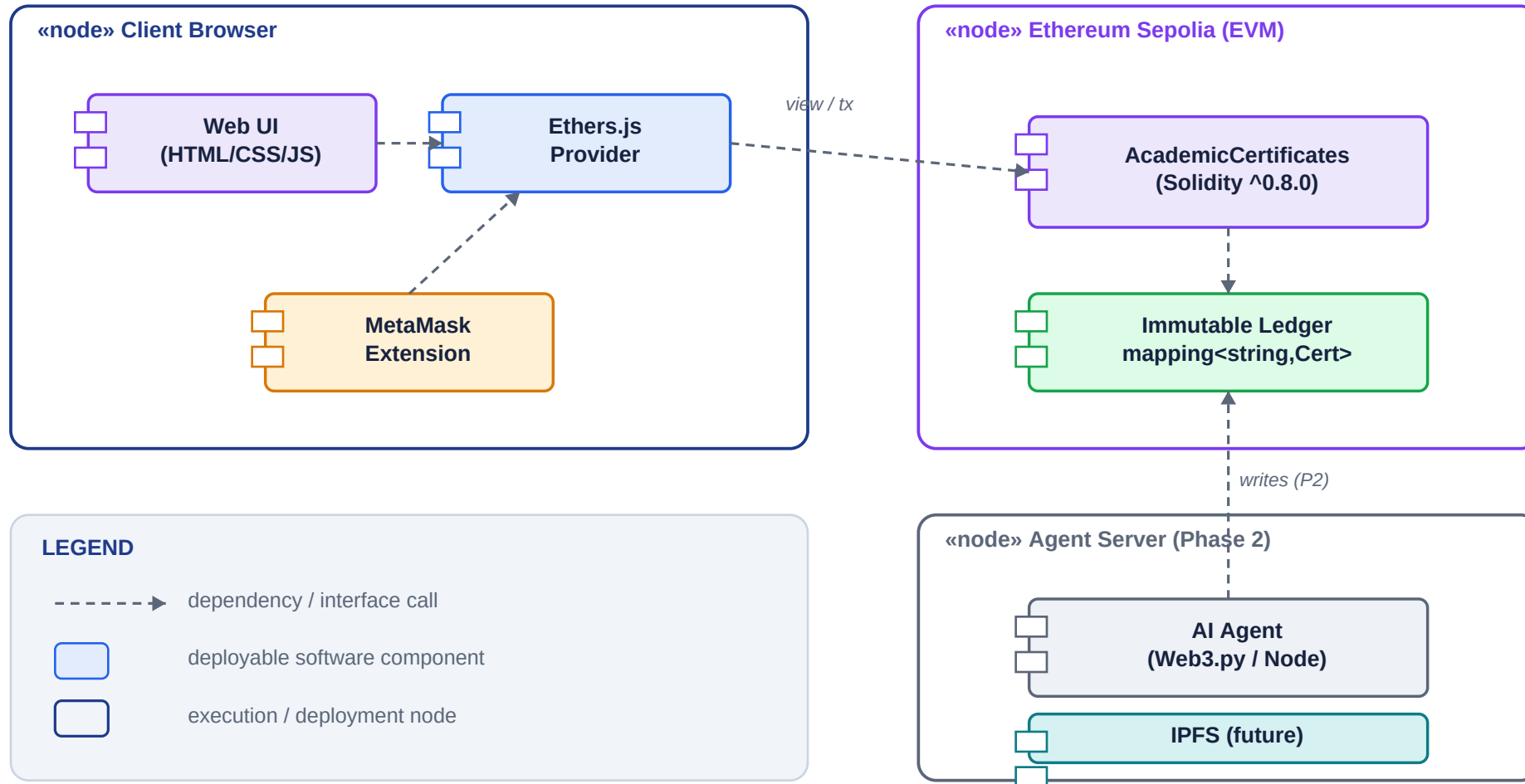


Verification (read path)



Component / Deployment Diagram

Components grouped by deployment node; dashed arrows are dependencies.



Algorithm Used in Proposed Solution

Algorithm 1 — issueDegree (WRITE)

Input : regNo, name, course, hash

Pre : caller wallet = university (RBAC)

1. require(msg.sender == university)
2. else revert // blocked at EVM
3. certificates[regNo] ←
Certificate(name, course, hash)

4. totalIssued ← totalIssued + 1

Cost : $O(1)$ write (+ gas in SepoliaETH)

Algorithm 2 — verifyCertificate (READ)

Input : regNo

1. data ← contract.certificates(regNo)
// gasless view call via Ethers.js
2. if data.studentName == "" then
3. render("Record Not Found") // .error
4. else
5. render(data) // .success

Cost : $O(1)$ read (0 gas, instant)

WHY $O(1)$?

A hash-table state mapping resolves the storage slot for a Registration Number directly, with no iteration over records — unlike array/loop designs that are $O(N)$ and whose gas cost grows with the number of students. RBAC require() rejects unauthorized writes before gas is spent.

Output Screenshots

Representative UI mock-ups — replace with your actual screenshots.

univerify.app/verify

UniVerify — Certificate Verification

Verify

✓ **Certificate Verified**

Name: <Student Name>

Course: B.Tech CSE

Hash: a1b2c3...(mock, P1)

univerify.app/verify

UniVerify — Certificate Verification

Verify

✗ **Record Not Found**

No certificate exists for this Registration Number. Zero-state result intercepted by the safety filter.

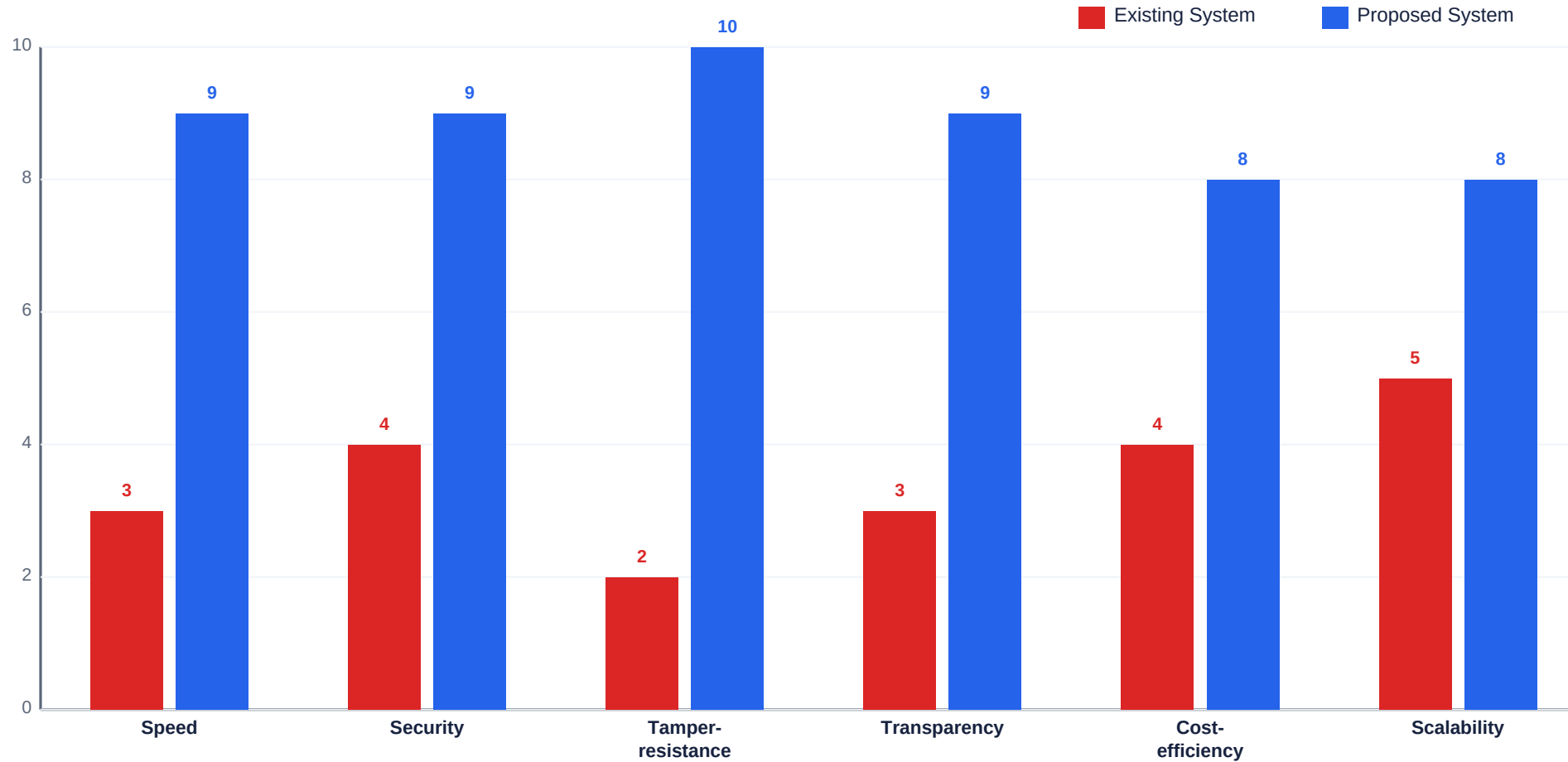
Remix IDE — issueDegree (Admin write, Phase 1)

transact (sign)

✓ status: 0x1 mined — gas used in SepoliaETH

Comparative Graph: Existing vs Proposed

Indicative qualitative scores (out of 10; higher is better).



Lower latency,
no SPOF, and
immutable on-
chain records
drive the gains.

Latency: ~7–14 days (existing) → ~seconds (proposed). Scores are indicative for comparison, not benchmarked units.

Future Enhancement & Conclusion

FUTURE ENHANCEMENTS

- **Phase 2 AI agent.** NLP-normalized ingestion, deterministic SHA-256 / IPFS hashing, programmatic transaction signing.
- **Mainnet / Layer-2.** Deploy on a low-fee L2 (e.g., Polygon) for production-scale, low-cost.
- **Revocation & expiry.** Support credential revocation and validity windows on-chain.
- **Multi-institution registry.** Federated admin roles for many universities.
- **QR + mobile DID wallet.** Scan-to-verify and self-sovereign holder wallets.

CONCLUSION

UniVerify V2.0 demonstrates that academic certificate verification can be transformed from a slow, centralized, forgery-prone process into an immutable, decentralized, constant-time lookup on a public blockchain.

The Phase-1 prototype validates the contract's state mutability, $O(1)$ string-keyed retrieval, RBAC guardrails, and gasless read path under a manual curation paradigm — with the data-fingerprint field held as a mock placeholder.

Phase 2 will close the loop with autonomous, cryptographically-hashed issuance, delivering an end-to-end trust-minimized credentialing system.

References

IEEE style — add your additional cited sources as needed.

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
- [2] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," White Paper, 2014.
- [3] J. Benet, "IPFS — Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.
- [4] MIT Media Lab, "Blockcerts: An Open Standard for Blockchain Certificates," 2016. [Online]. Available: <https://www.blockcerts.org>
- [5] A. Grech and A. F. Camilleri, "Blockchain in Education," Joint Research Centre (JRC), European Commission, 2017.
- [6] Ethereum Foundation, "Solidity Documentation (v0.8.x)," [Online]. Available: <https://docs.soliditylang.org>
- [7] Ethers.js, "Documentation (v5.7)," [Online]. Available: <https://docs.ethers.org/v5/>
- [8] ConsenSys, "MetaMask Developer Documentation," [Online]. Available: <https://docs.metamask.io>

Note: references [1]–[5] are foundational works; replace or augment with the specific papers cited in your report.